COLECCIONEX

DOCUMENTACIÓN DEL PROYECTO

COLECCIONEX

Plataforma web para la gestión y comercialización de cartas coleccionables online

Equipo de desarrollo: Uziel Bargas · Aaron Cuevas

Materia: Taller de Construcción de Software — EPET N.º 20

Versión: 2.0 (versión final para la Expo)

Última actualización: 8 de noviembre de 2026

Repositorio: github.com/UzielBargas/Coleccionex

Índice

1. Historial de cambios
2. Etapa de análisis
    1. Descripción general
    2. Requisitos de usuario
    3. Requisitos de sistema
    4. Requisitos no funcionales
    5. Actores y casos de uso
    6. Modelo de datos (DER)
    7. Diagramas de secuencia y de casos de uso
3. Etapa de diseño
    1. Mapa de navegación
    2. Prototipos de interfaz
    3. Identidad visual
4. Implementación
    1. Tecnologías utilizadas
    2. Arquitectura
    3. Estructura de carpetas
    4. Base de datos
    5. API (endpoints)
    6. Reglas de negocio
    7. Seguridad
    8. Pantallas finales
5. Instalación y ejecución
6. Diferencias con el análisis y trabajo futuro

1. Historial de cambios

Versión	Fecha	Cambios
1.0	6 de septiembre de 2026	Etapa de análisis y diseño: requisitos, actores, casos de uso, DER, diagramas de secuencia y de clases, mapa de navegación y prototipos de interfaz.
1.1	22 de octubre de 2026	Corrección del profesor: tipografía unificada, vista de cartas y vista de mazos. Primera versión de la base de datos y del inicio de sesión.
2.0	8 de noviembre de 2026	Implementación completa: mercado entre usuarios en **pesos argentinos**, publicación de cartas con foto, carga de saldo, compras con comisión, reportes, historial, mazos, modo visitante y **panel de administrador**. Nuevo modelo de datos de 7 tablas. Se actualizan los requisitos para reflejar lo que se construyó (ver capítulo 6).

2. Etapa de análisis

2.1 Descripción general

Coleccionex es una página web de cartas coleccionables de series, juegos y deportes (Yu-Gi-Oh!, Pokémon, Magic: The Gathering y figuritas de fútbol, entre otras). El cliente puede **vender, comprar y buscar** las cartas que desee.

Para operar tiene que registrarse e iniciar sesión. Así tiene un **inventario** con sus cartas y puede armar **mazos**. Hay un apartado donde elige el método de pago para **cargar saldo en pesos argentinos**; por ejemplo, el usuario carga $ 50.000 pagando con Mercado Pago, Ualá o Naranja X.

Si decide vender, sube una foto de la carta y especifica su estado, medidas, peso, año y la colección a la que pertenece. Cuando otro usuario la compra, el precio se descuenta del saldo del comprador, la carta pasa a su inventario y el vendedor cobra el precio menos una comisión del 5% para la plataforma.

El comprador puede **reportar al vendedor** si comete una infracción; por ejemplo, vender una carta publicada como "Nuevo" que está en un estado deplorable. Todas las compras, ventas y cargas de saldo quedan en un **historial de transacciones**.

Un **administrador** recibe los reportes de los usuarios y puede tomar medidas. Puede inspeccionar las cuentas: ve el historial reciente, las cartas y los reportes del usuario denunciado. Si encuentra actividades sospechosas o ilegales, lo **banea**.

2.2 Requisitos de usuario

Código	Requisito	Estado
RU-1	El usuario debe poder registrarse e iniciar sesión en la plataforma.	**Cumplido**
RU-2	Se debe poder gestionar el perfil (agregar, modificar o actualizar datos). Para operar con dinero, los datos tienen que estar completos.	**Cumplido**
RU-3	El usuario debe poder publicar cartas para la venta indicando sus especificaciones.	**Cumplido**
RU-4	El sistema debe validar la identidad del usuario antes de cargar saldo con los métodos disponibles (Mercado Pago, Ualá, Naranja X), para evitar que personas ajenas operen en su nombre.	**Parcial** (pago simulado)
RU-5	Para comprar, el sistema debe verificar que el usuario inició sesión y que sus datos están completos.	**Cumplido**
RU-6	El usuario debe poder buscar cartas sin registrarse, pero de forma limitada. Para ver todo, debe registrarse o iniciar sesión.	**Cumplido**
RU-7	El usuario debe poder reportar vendedores por diversas razones (estado distinto al publicado, precio abusivo, etc.).	**Cumplido**
RU-8	El usuario debe poder armar mazos con sus cartas, verlos y disolverlos en cualquier momento. Un mazo se considera completo con al menos 10 cartas.	**Cumplido**

RU-9	El administrador debe poder ver el historial de transacciones y los reportes de los usuarios para resolverlos, aplicando el baneo en caso de fraude o infracción.	**Cumplido**

2.3 Requisitos de sistema

Código	Requisito	Cómo se cumple en la implementación
RS-1 (RU-1)	Validar las credenciales antes de permitir el inicio de sesión.	`api/login.php` busca por usuario o correo y compara con `password_verify`. Las cuentas baneadas no pueden entrar.
RS-2 (RU-1)	Validar los campos obligatorios del registro antes de crear la cuenta.	El formulario usa `required` y `minlength`. `api/registro.php` vuelve a validar usuario, correo, contraseña de 8 caracteres o más y la confirmación, y avisa si el usuario o el correo ya existen.
RS-3 (RU-2)	Restringir funciones del panel hasta que el usuario complete sus datos.	Mientras el perfil no tenga nombre y teléfono, se muestra un aviso en el Inicio y PHP bloquea la compra y la carga de saldo (`exigirPerfilCompleto`). Publicar, armar mazos y buscar quedan habilitados.
RS-4 (RU-3)	Exigir estado, medidas, peso, año, colección y al menos una foto para publicar.	`api/subir_carta.php` valida cada campo y que la foto sea una imagen real (JPG, PNG o WEBP de hasta 3 MB).
RS-5 (RU-4)	Avisar al usuario después de cada movimiento de saldo.	Cada movimiento se muestra en pantalla al momento y queda registrado en el historial. *El envío de correo o SMS queda como trabajo futuro.*
RS-6 (RU-5)	Interceptar el botón de compra: sin sesión o sin datos completos, redirigir al login o al perfil con un aviso.	El visitante ve "Iniciá sesión para comprar". Con sesión, la confirmación revisa el perfil y el saldo antes de comprar, y PHP lo vuelve a controlar.
RS-7 (RU-6)	Buscar por coincidencia de texto (comienzo, medio o fin) y por categorías de colección.	`api/publicaciones.php` usa `LIKE '%texto%'` sobre el nombre de la carta y de la colección, con filtros por juego, colección, estado, año y precio.
RS-8 (RU-7)	Formulario de denuncia en la carta, con motivo y descripción.	El detalle de la carta tiene "Reportar vendedor", con 5 motivos y una descripción de 10 a 500 caracteres (`api/reportar.php`).
RS-9 (RU-8)	Agrupar cartas en mazos, verlos y disolverlos.	Cada carta puede estar en un solo mazo. La lista muestra el progreso hasta 10 cartas, y "Disolver mazo" devuelve las cartas al inventario.
RS-10 (RU-9)	Vista exclusiva del administrador: historial, reportes y baneo.	Pestaña "Administración", visible solo con `rol = 'admin'`. Cada pedido se controla en PHP con `requiere_admin.php`.

2.4 Requisitos no funcionales

- **Seguridad:** las contraseñas se guardan cifradas, todas las consultas son preparadas y cada operación de dinero se hace dentro de una transacción de la base de datos (ver 4.7).

- **Rendimiento:** la búsqueda espera a que el usuario termine de escribir antes de consultar, y solo se piden los datos de la sección que se usa.
- **Usabilidad:** el Inicio explica qué se puede hacer. Hay avisos en pantalla después de cada acción, mensajes de ayuda cuando una sección está vacía y confirmación antes de comprar, borrar o banear.
- **Compatibilidad:** funciona en Chrome, Edge y Firefox, y se adapta a pantallas de computadora y de celular.

2.5 Actores y casos de uso

Actor	Descripción	Casos de uso
Visitante	Persona que entra sin iniciar sesión.	Ver el Inicio · Buscar cartas (hasta 8 resultados) · Ver el detalle de una carta · Registrarse · Iniciar sesión
Cliente	Usuario registrado que compra, vende, gestiona su inventario y carga saldo.	Gestionar perfil · Cargar saldo · Publicar carta · Buscar y filtrar · Comprar carta · Gestionar inventario y mazos · Reportar vendedor · Consultar historial
Administrador	Usuario interno que gestiona reportes, inspecciona historiales y aplica baneos.	Revisar bandeja de reportes · Inspeccionar usuario (historial, cartas y reportes) · Banear y quitar baneo · Marcar reportes como revisados

2.6 Modelo de datos (DER)

El modelo final tiene 7 tablas. Reemplaza al DER y al diagrama de clases de la versión 1.0: la implementación no usa clases sino funciones de PHP, y cada tabla corresponde a una entidad del sistema.

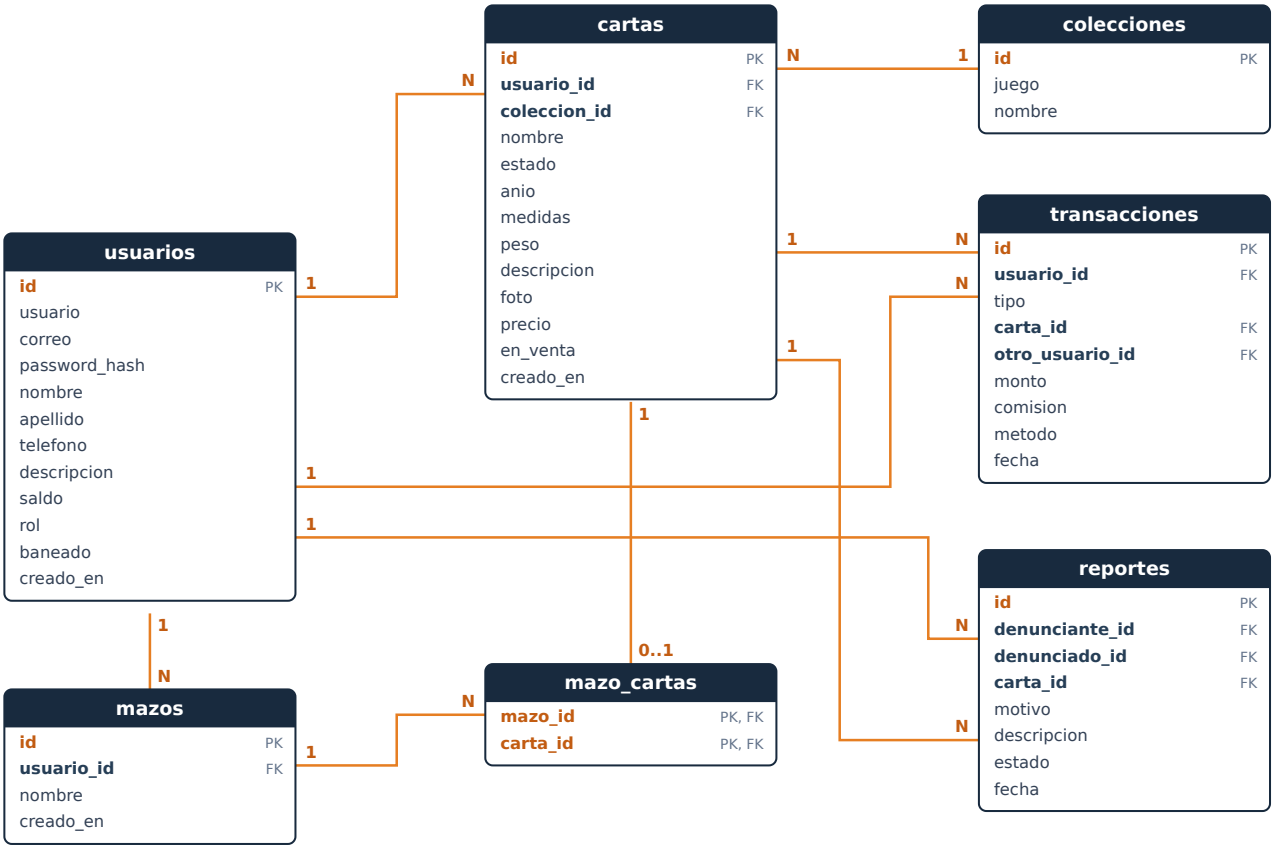


Figura 1 — Diagrama entidad-relación de la base de datos `coleccionex` (PK = clave primaria, FK = clave foránea).

Tabla	Qué guarda
`usuarios`	Cuentas de clientes y administradores: datos de perfil, contraseña cifrada, saldo en pesos, rol y si está baneado.
`colecciones`	Juego y edición a la que pertenece una carta (por ejemplo, Pokémon — Base Set).
`cartas`	Cada fila es una carta física con su dueño actual, estado, año, medidas, peso, foto, precio y si está a la venta.
`mazos` y `mazo_cartas`	Los mazos de cada usuario y qué cartas contienen. Una carta está como máximo en un mazo.
`transacciones`	Compras, ventas (con la comisión) y cargas de saldo (con el método de pago).
`reportes`	Denuncias de un usuario a otro: motivo, descripción, carta y estado (pendiente o revisado).

2.7 Diagramas de secuencia y de casos de uso

Los diagramas de la etapa de análisis siguen representando el funcionamiento implementado. "ControladorReportes" y "ControladorCompras" corresponden a `api/reportar.php` y `api/comprar.php`.

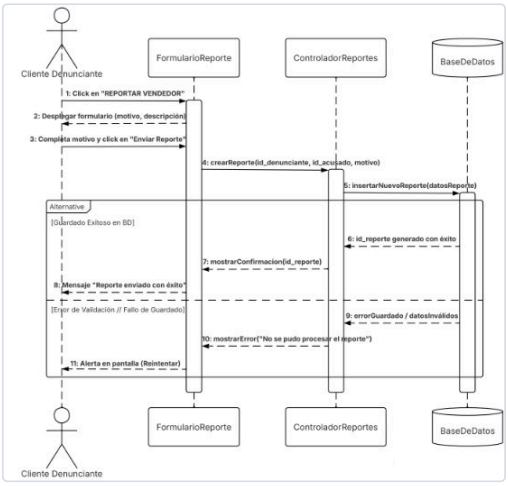


Figura 2 — Secuencia: reportar a un vendedor.

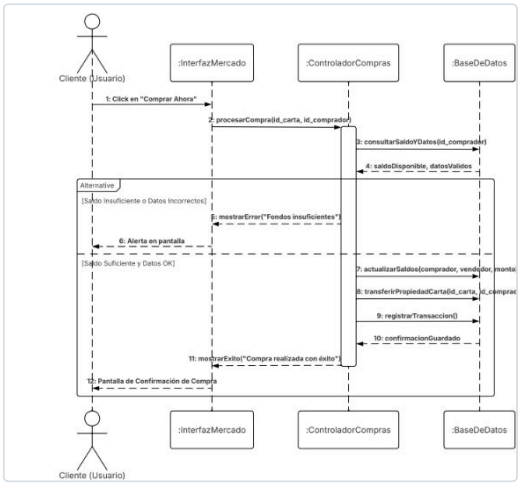


Figura 3 — Secuencia: comprar una carta.

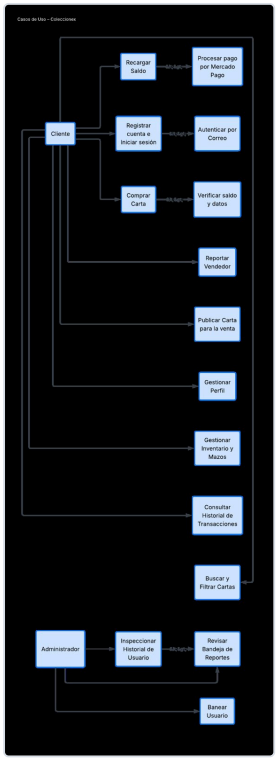


Figura 4 — Diagrama de casos de uso (versión 1.0).

3. Etapa de diseño

3.1 Mapa de navegación

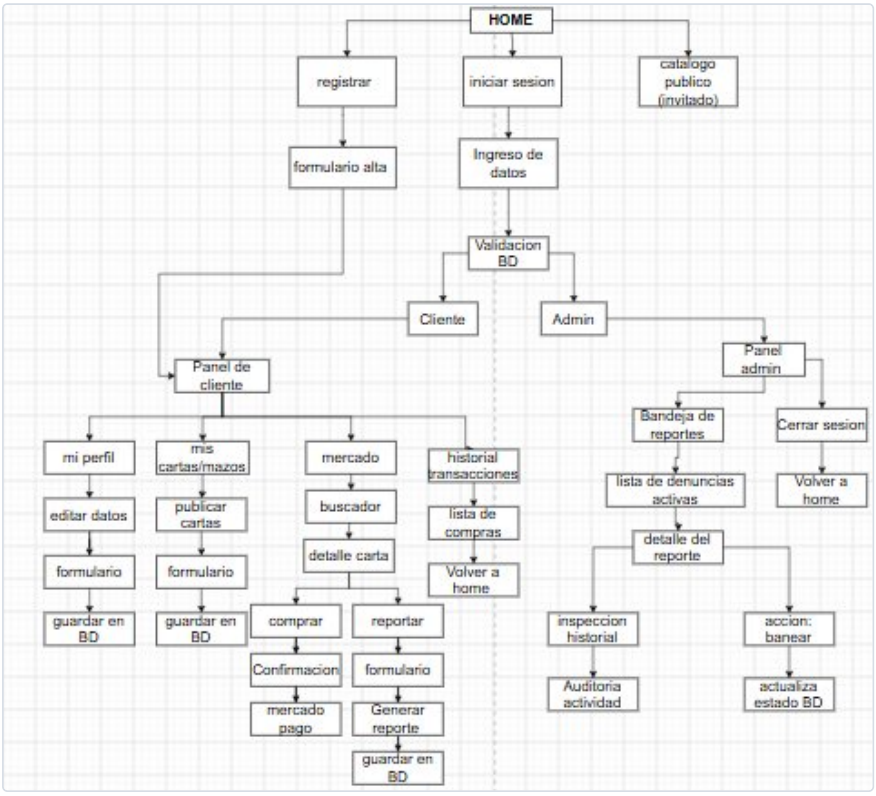


Figura 5 — Mapa de navegación de la etapa de diseño.

La navegación implementada sigue ese mapa con un menú de pestañas en una sola página (`index.html`):

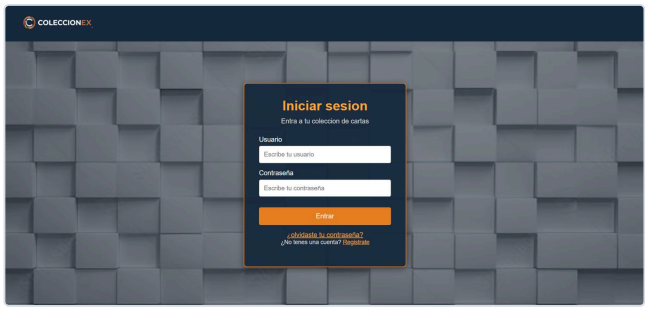
- **Visitante:** Inicio · Mercado. Arriba: Registrarse · Iniciar sesión.
- **Cliente:** Inicio · Mi perfil · Mis cartas / mazos · Mercado · Historial. Arriba: saldo y Cerrar sesión.
- **Administrador:** lo mismo que el cliente, más la pestaña **Administración**.
- **Ventanas emergentes:** detalle de carta (comprar, reportar, publicar, mazo), Publicar carta e Inspeccionar usuario.
- **Páginas aparte:** `paginas/sesion.html`, `paginas/registro.html` y `paginas/sobre-nosotros.html`.

3.2 Prototipos de interfaz

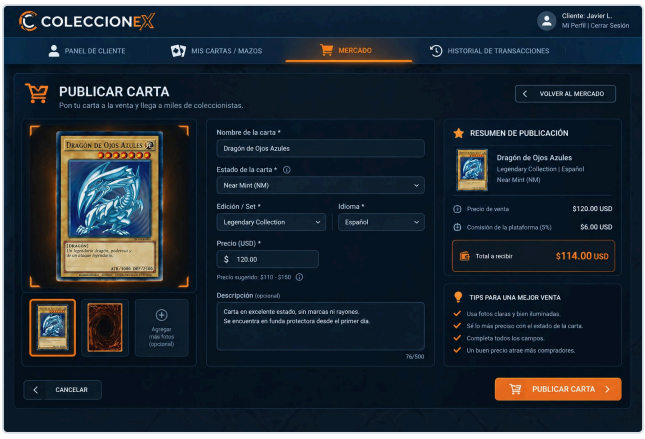
Estos fueron los prototipos de la etapa de diseño. La versión final mantiene su estructura (encabezado con logo y saldo, menú de pestañas, tarjetas de cartas y resumen al publicar) y los adapta a la identidad visual de Coleccionex. Las pantallas finales están en el punto 4.8.



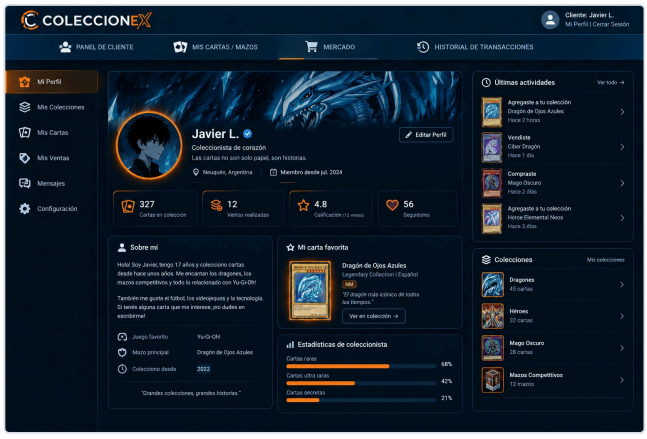
Home pública



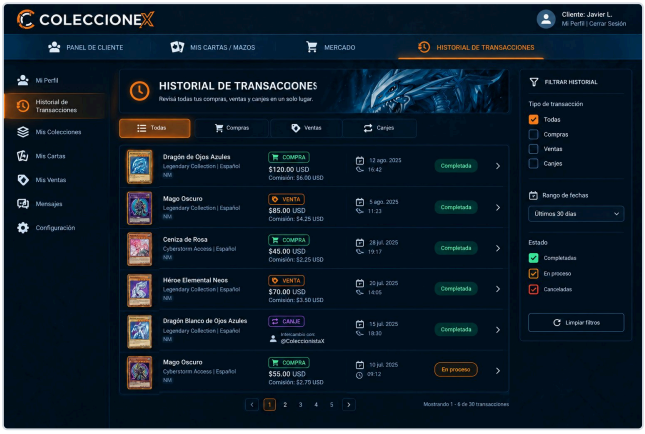
Inicio de sesión / registro



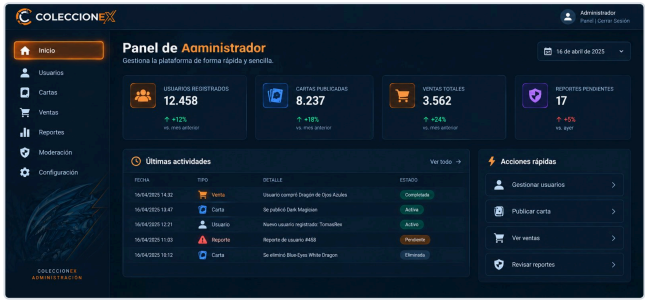
Publicar carta



Perfil de usuario



Historial de transacciones



Panel de administrador

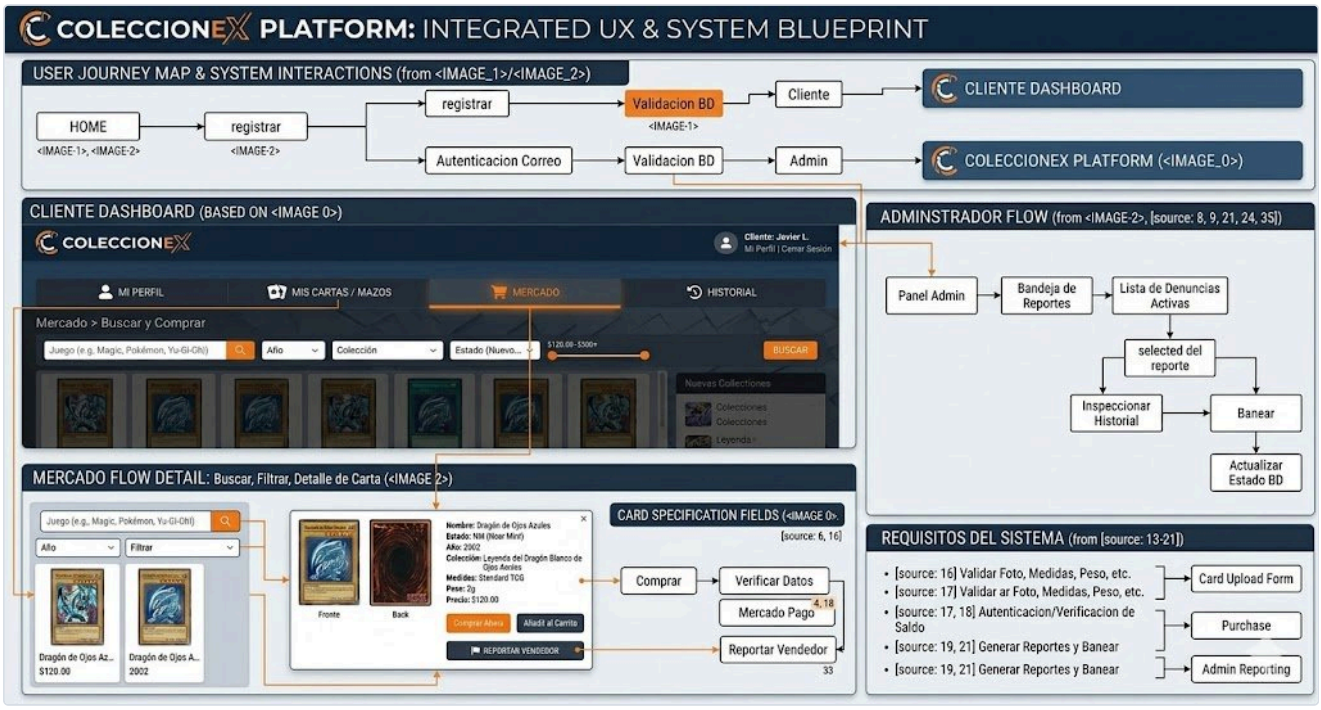


Figura 6 — Mapa de experiencia de usuario (UX).

3.3 Identidad visual

Elemento	Decisión
Colores	Azul oscuro `#182A3E` (encabezado y pie), azul medio `#263E56` (menú y cajas) y naranja `#E67E22` (botones y elementos activos). Estos colores se definen una sola vez como variables en `css/Estilos.css`.
Tipografía	Una sola fuente para títulos y botones (Rajdhani) y una para textos (Arial), en todas las páginas. Así se resolvió la corrección del profesor sobre la tipografía.
Carta (`.card-box`)	Proporción de carta real (63 × 88 mm), foto, nombre, colección y año, etiqueta de estado y precio en pesos. El color del marco depende del juego.
Mazos	Lista a la izquierda con una barra de progreso hasta 10 cartas, detalle arriba, inventario abajo y una columna de "cartas sin mazo".
Avisos	Mensaje flotante verde (éxito) o rojo (error) después de cada acción, y carteles en las secciones vacías explicando qué hacer.

4. Implementación

4.1 Tecnologías utilizadas

Parte	Tecnología
Estructura	HTML5
Estilos	CSS3: Grid, Flexbox, variables y *media queries* para celular
Lógica del navegador	JavaScript sin librerías: `fetch`, `FormData`, `createElement` y `Intl.NumberFormat` para los pesos
Servidor	PHP 8 con PDO, sesiones, `password_hash` y subida de archivos
Base de datos	MySQL / MariaDB (incluida en XAMPP), administrada con phpMyAdmin
Formato de intercambio	JSON
Herramientas	XAMPP, Visual Studio Code, Git y GitHub

4.2 Arquitectura

La aplicación tiene tres capas. El navegador nunca habla directo con la base de datos: le pide a un archivo PHP de la carpeta `api/`, que controla la sesión y los datos, consulta MySQL y responde en JSON. Es el mismo esquema que propone la guía de la cátedra.

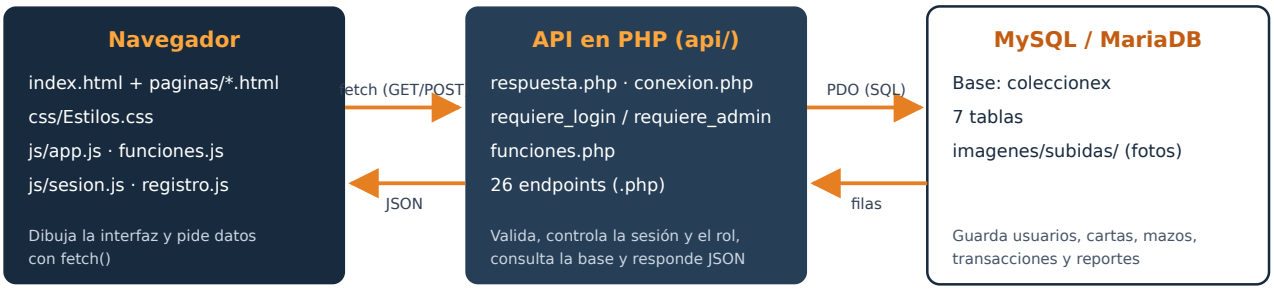


Figura 7 — Arquitectura del sistema.

4.3 Estructura de carpetas

Carpeta / archivo	Contenido
`index.html`	Página principal: Inicio, Mi perfil, Mis cartas / mazos, Mercado, Historial, Administración y las ventanas emergentes.
`paginas/`	`sesion.html`, `registro.html`, `sobre-nosotros.html`
`css/`	`Estilos.css` (principal), `sesion.css`, `sobre-nosotros.css`

`js/`	`funciones.js` (cambio de pestañas), `app.js` (toda la lógica de la página), `sesion.js` y `registro.js`
`api/`	26 archivos PHP que responden JSON, más los archivos compartidos `conexion.php`, `respuesta.php`, `funciones.php`, `requiere_login.php` y `requiere_admin.php`
`imagenes/`	Logo, fondos, `cartas/` (fotos de ejemplo) y `subidas/` (fotos que suben los usuarios)
`src/coleccionex.sql`	Script que crea la base, las tablas y los datos de ejemplo

4.4 Base de datos

El script `src/coleccionex.sql` crea la base `coleccionex` con codificación `utf8mb4`, las 7 tablas del DER con sus claves foráneas y datos de ejemplo:

- 2 vendedores de prueba y 16 cartas a la venta, de 7 colecciones y 4 juegos.
- 1 cuenta de administrador.
- Algunos movimientos y 1 reporte pendiente, para que el panel de administrador no arranque vacío.

Las claves foráneas usan `ON DELETE CASCADE` o `SET NULL` según el caso. Por ejemplo, si se borra un mazo se borran sus filas de `mazo_cartas`, pero las cartas siguen en el inventario.

4.5 API (endpoints)

Archivo	Método	Qué hace
`registro.php`	POST	Crea un usuario e inicia su sesión
`login.php` / `logout.php`	POST	Inicia (con usuario o correo) y cierra la sesión
`sesion.php`	GET	Datos del usuario conectado: perfil, saldo, rol y números
`perfil.php`	POST	Modifica nombre, apellido, teléfono, correo y descripción
`recargar.php`	POST	Carga saldo (simulado) con Mercado Pago, Ualá o Naranja X
`colecciones.php`	GET	Lista de colecciones (pública)
`publicaciones.php`	GET	Mercado con buscador y filtros (pública, limitada a 8 sin sesión)
`mis_cartas.php`	GET	Inventario del usuario
`subir_carta.php`	POST	Sube una carta con foto y opcionalmente la publica
`cambiar_venta.php` / `borrar_carta.php`	POST	Publica o pausa la venta de una carta / la elimina
`comprar.php`	POST	Compra una carta (transacción completa)
`reportar.php`	POST	Reporta a un vendedor
`historial.php`	GET	Movimientos del usuario, con filtro por tipo

`mazos.php` / `mazo.php`	GET	Mazos del usuario / cartas de un mazo
`crear_mazo.php`, `borrar_mazo.php`, `agregar_a_mazo.php`, `quitar_de_mazo.php`	POST	Gestión de mazos
`admin_reportes.php`	GET	Bandeja de reportes y resumen (solo administrador)
`admin_usuarios.php` / `admin_usuario.php`	GET	Buscar usuarios / inspeccionar uno: historial, cartas y reportes
`admin_resolver.php` / `admin_banear.php`	POST	Marcar un reporte como revisado / banear o quitar el baneo

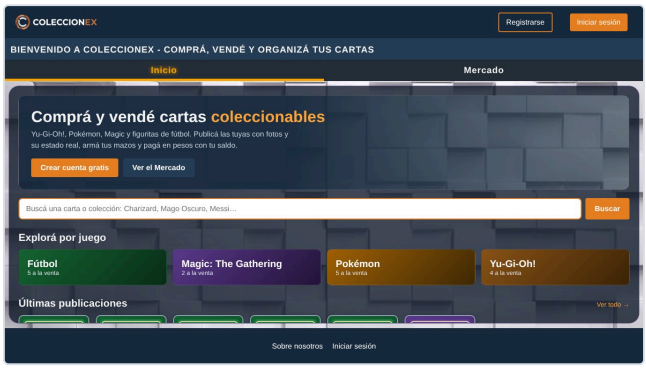
4.6 Reglas de negocio

- **Compra:** se descuenta el precio solo si al comprador le alcanza el saldo. La carta cambia de dueño, sale de cualquier mazo del vendedor y deja de estar a la venta. El vendedor recibe el precio menos el **5% de comisión**. Todo ocurre dentro de una transacción (`beginTransaction` / `commit`): si un paso falla, no se cobra nada.
- **Doble compra:** si dos usuarios compran la misma carta al mismo tiempo, solo se completa la primera compra; la segunda recibe "Alguien la compró antes que vos".
- No se puede comprar una carta propia ni reportarse a uno mismo.
- **Carga de saldo:** entre $ 500 y $ 1.000.000 por operación.
- **Baneo:** la cuenta no puede iniciar sesión, su sesión abierta se corta, sus cartas salen del Mercado y sus reportes pendientes quedan revisados. Un administrador no puede banearse a sí mismo ni a otro administrador.

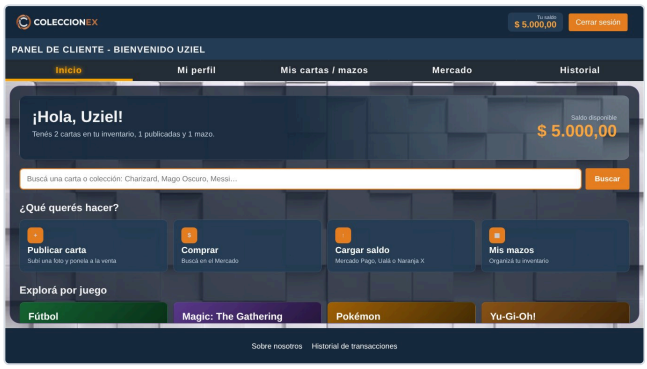
4.7 Seguridad

- Las contraseñas se guardan con `password_hash` y se verifican con `password_verify`. Al iniciar sesión se regenera el identificador de sesión.
- Todas las consultas usan sentencias preparadas de PDO, que evitan la inyección SQL. Los valores que no pueden ir como parámetro (orden y límite) se toman de una lista cerrada.
- Los endpoints privados incluyen `requiere_login.php`, y los del panel, `requiere_admin.php`. Cada operación controla que la carta o el mazo pertenezcan al usuario conectado.
- Las fotos se validan por su contenido (`getimagesize`), no por la extensión, y se guardan con un nombre al azar. Un `.htaccess` impide ejecutar PHP en `imagenes/subidas/`.
- JavaScript arma la pantalla con `textContent`, por lo que un texto escrito por un usuario no se ejecuta como código.
- Los errores internos se registran con `error_log` y al usuario solo se le muestra un mensaje simple.

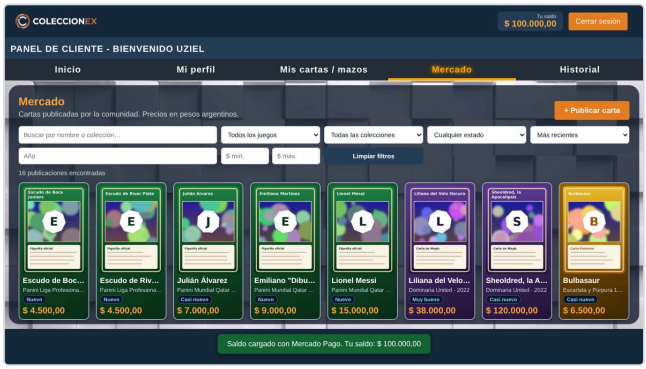
4.8 Pantallas finales



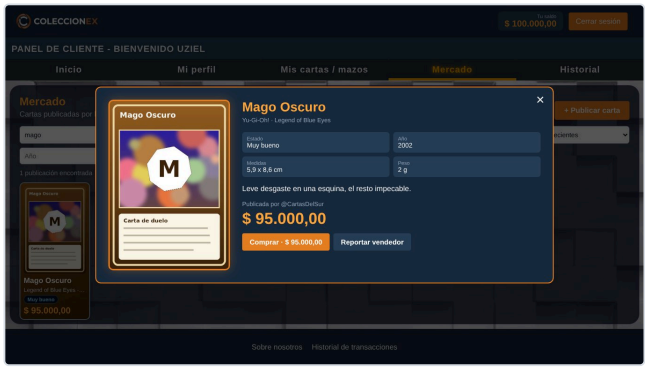
Inicio (visitante)



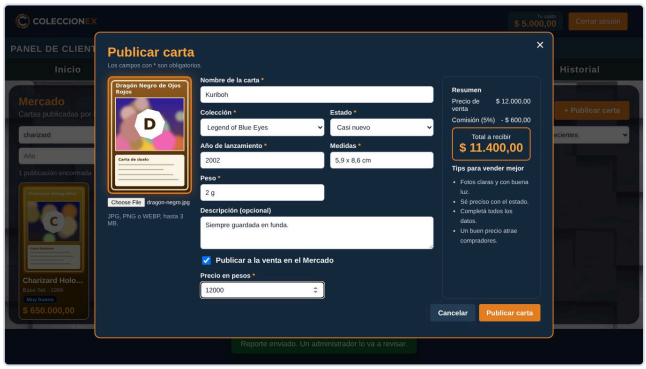
Inicio (cliente)



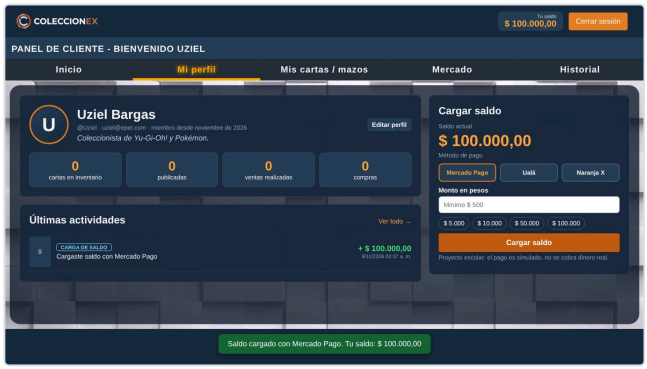
Mercado



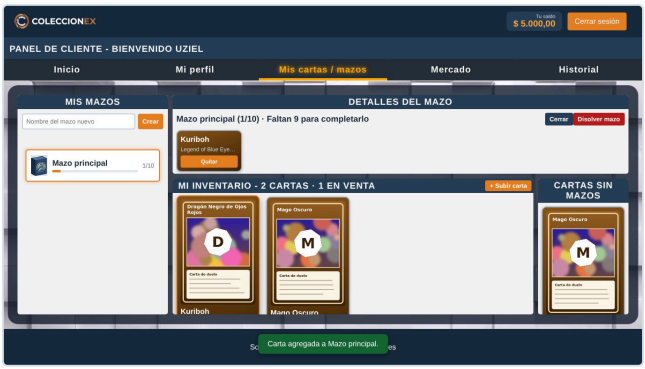
Detalle de carta



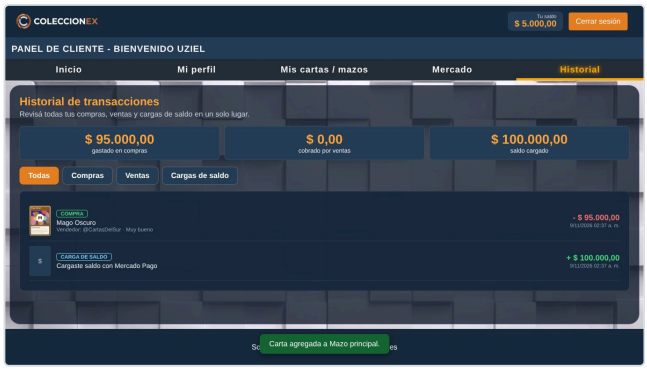
Publicar carta



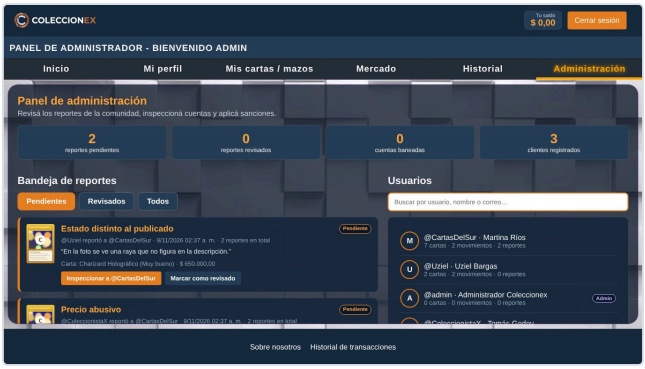
Mi perfil y carga de saldo



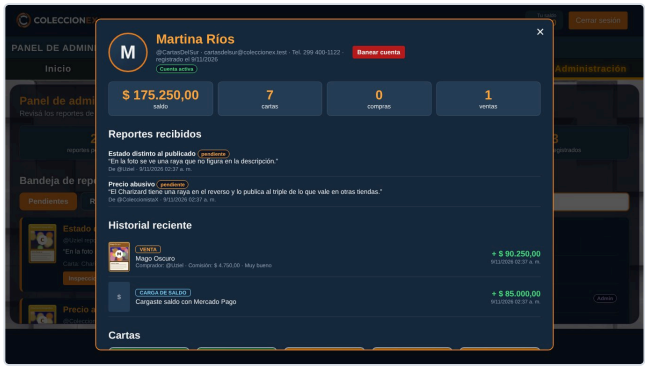
Mis cartas / mazos



Historial de transacciones



Panel de administración



Inspeccionar usuario

5. Instalación y ejecución

Herramientas necesarias: XAMPP 8.0 o superior (Apache, PHP 8, MySQL/MariaDB y phpMyAdmin) y un navegador moderno. De forma opcional, Visual Studio Code y Git.

1. Instalar XAMPP, abrir el **XAMPP Control Panel** e iniciar **Apache** y **MySQL**.
2. Copiar la carpeta del proyecto dentro de `C:\xampp\htdocs\`, de modo que quede `C:\xampp\htdocs\Coleccionex\index.html`.
3. Entrar a `http://localhost/phpmyadmin`, abrir la pestaña **SQL**, pegar el contenido de `src/coleccionex.sql` y presionar **Continuar**.
4. Abrir `http://localhost/Coleccionex/` en el navegador. No abrir el archivo con doble clic, porque PHP no se ejecuta.

Cuentas de prueba

Usuario	Contraseña	Rol
`admin`	`admin1234`	Administrador
`CartasDelSur`	`demo1234`	Cliente con cartas a la venta
`ColeccionistaX`	`demo1234`	Cliente con cartas a la venta

Las instrucciones detalladas y los problemas comunes están en el `README.md` del repositorio.

6. Diferencias con el análisis y trabajo futuro

En el análisis (v1.0)	En la implementación (v2.0)
Saldo en dólares	Saldo y precios en **pesos argentinos**.
Validación del inicio de sesión por correo electrónico	Validación con usuario (o correo) y contraseña cifrada. El correo de validación queda como trabajo futuro.
Alertas por correo o SMS en cada movimiento de saldo	Aviso en pantalla y registro en el historial. El envío de correo o SMS queda como trabajo futuro.
Pago real con Mercado Pago, Ualá o Naranja X	Carga de saldo **simulada**; en la página se aclara que no se cobra dinero real.
Mazo creado solo al alcanzar 10 cartas	El mazo se crea vacío y figura como "completo" a partir de 10 cartas. Así se puede armar de a poco y se ve el progreso.
Bloqueo de todo el panel hasta confirmar los datos	Se bloquean solo las operaciones con dinero (comprar y cargar saldo) hasta completar nombre y teléfono, con un aviso en el Inicio.
Diagrama de clases	Reemplazado por el DER (2.6) y la arquitectura en capas (4.2), porque el código está organizado en funciones de PHP y JavaScript.
—	Se agregaron la comisión del 5%, el modo visitante, la búsqueda de usuarios en el panel de administración y "quitar baneo".

Trabajo futuro

- Integrar el pago real con las billeteras virtuales y confirmar la acreditación del saldo.
- Enviar correos de verificación de cuenta y alertas de movimientos.
- Permitir varias fotos por carta y un chat entre comprador y vendedor para coordinar la entrega.
- Retiro del saldo a una cuenta bancaria o billetera.